Personal Statement

The Hong Kong University of Science and Technology
Jianming Xing MSc in Information Technology

A learned robot policy becomes useful only when data, software, and hardware agree about what the robot sees and should do. In my CR5 pipeline, timestamp alignment, consistent state-action fields, and structural and motion checks made episodes coherent inputs; these implementation choices did not by themselves alter physical behavior. Physical failure arose during execution: under non-blocking operation, a new inference could consume an intermediate in-motion state before the prior action chunk finished, driving back-and-forth corrections. That distinction shifted my focus from isolated prediction to connecting representation, learning, serving, and action.

To gain end-to-end competence across machines and computation, I built my education as a bridge rather than a replacement. In June 2025, I earned a B.Eng. in Mechanical Design, Manufacturing and Automation at Harbin Institute of Technology, Weihai. That training grounded me in mechanism behavior, kinematics, and physical constraints. I am now pursuing a second bachelor's degree in Computer Science and Technology at Harbin Institute of Technology, Shenzhen, expected in June 2027. This expansion lets me reason both about a machine's physical capabilities and about how algorithms, data structures, interfaces, and runtime decisions determine its behavior. Together, these disciplines help me follow a learned capability from representation to physical execution.

My research examines how action representation shapes what a vision-language-action model can learn. As a Research Assistant at iLearn, HIT Shenzhen, I am second author of Hermite Action Tokenizer for VLA, an unsubmitted working paper, and I am responsible for its discrete autoregressive tokenization branch. I worked on splitting seven-dimensional action chunks into six-dimensional end-effector trajectories and gripper states, using shared breakpoints to remove redundant endpoints and quantization seams while coding gripper changes separately. This preserved trajectory continuity without forcing discrete gripper states into the same curve. I also compared adaptive MLP-based and fixed Hermite segmentation. This design raised a central evaluation question: how can I isolate the quality of an encoding from the behavior of a complete policy? In an offline codec benchmark averaging four LIBERO subsets with 8-by-7 action chunks, Hermite's Endpoint MSE was approximately 99.7% lower than FAST and 82.6% lower than BEAST. The comparison demonstrates codec fidelity under that protocol; it does not establish online rollout success or physical-robot reliability.

Separate systems work on a Dobot CR5 showed how representation choices travel through an implemented learning loop. I anchored HDF5 records to RGB timestamps and aligned robot states, target actions, and gripper feedback through binary search and linear interpolation. Before downstream use, I checked schemas, timing, motion, and workspace constraints, then converted the episodes into LeRobot v2.0 with consistent seven-dimensional robot-plus-gripper states and actions. I adapted OpenPI through a training configuration, WebSocket policy serving, and a CR5 client that assembled observations, received action chunks through the server, and translated each seven-dimensional action into a six-component ServoP arm target and one Modbus gripper command. In my tests, non-blocking execution increased apparent command frequency, but a new inference could use an intermediate in-motion state before the preceding action chunk had finished, producing back-and-forth corrections. I therefore used blocking, short-horizon ServoP chunks, accepting lower apparent frequency to preserve state-action consistency. This was an execution decision rather than a change in model score. Although Hermite and CR5 were separate efforts, together they taught me to connect a representation

question with engineering questions about clocks, software interfaces, observations, and action execution.

Supporting projects extended this perspective. For a weld-seam HMI, I connected C++/Qt controls to Python/PyTorch segmentation inference through QProcess and visualized robot kinematics in OpenGL, keeping the desktop interface, inference process, and robot model as intelligible components. During the Xbotics Embodied AI Community Internship, I migrated an ANYmal-C navigation task from Isaac Lab into a MotrixLab NumPy environment. I refactored reset and step flows, corrected coordinate handling, and used rollout and reward/termination regression checks to examine whether the migrated environment retained its intended behavior. Both projects made cross-component change a concrete software-design problem.

My immediate goal is AI/ML or learning-systems R&D for embodied agents, building methods that remain dependable when connected to real data and hardware. I want graduate training to deepen my foundations in machine learning, data systems, and scalable software while testing them against sensing and control constraints. After gaining stronger real-system experience, I may consider doctoral study as an option rather than a predetermined step. I seek curricular depth and applied opportunities to turn this perspective into rigorous, transferable engineering. What I lack is a software architecture that can evolve without breaking the meaning of data passed among recognition, learning, serving, and robot execution. My ROS2-based CR5 pipeline connected all four, but each handoff was implemented separately and a change in one component could surface much later as unexpected motion. I want to learn how reliable software design can make such changes easier to manage and diagnose. HKUST's MSc in Information Technology offers the software and learning depth for that work.

With the current courses available in my intake and CSIT6910 Independent Project approved, CSIT5100 Engineering Reliable Software Systems would guide how I structure change, testing, and fault localization. CSIT5410 Recognition Systems and CSIT5910 Machine Learning would strengthen the perceptual inputs and learned outputs around that software core. In CSIT6910, I could build a versioned pipeline in which a revised sensor representation or model is propagated through WebSocket serving to robot command, then use targeted tests to locate where behavior changes. Developing that maintainable system would prepare me to engineer embodied-agent stacks whose sensing, learning, and execution can evolve together in robotics R&D.